1 Лекция 01. Параметр как часть постановки: FPT и XP

1.1 Зачем нужен параметр

Предположим, большой граф описывает конфликты: ребро означает, что две вершины нельзя оставить одновременно. Разрешено удалить не более k вершин, чтобы конфликтов не осталось. Граф может содержать миллион вершин, а допустимое число удалений — десять. Обычная оценка «задача NP-полна» не различает такой вход и вход, на котором можно удалить половину графа. Параметризованный анализ сохраняет в оценке две величины: размер данных и выбранную характеристику трудности.

Эта лекция вводит параметризованную задачу, классы FPT и XP и объясняет их на двух графовых задачах. Нужны понятия алгоритма, полиномиального времени и NP-полноты. Новые определения ниже не требуют просмотра записи.

1.2 Соглашения и постановки

Граф $G = (V, E)$ конечный, простой и неориентированный: петли и кратные рёбра отсутствуют. Пишем $n = |V|$, $m = |E|$. Вершины u, v смежны, если $uv \in E$. Подмножество $S \subseteq V$ называется **вершинным покрытием (vertex cover)**, если у каждого ребра хотя бы один конец принадлежит S. Его удаление убирает все рёбра. Задача **Vertex Cover** получает (G, k) и спрашивает, существует ли такое S размера не больше k.

Например, для пути $a - b - c - d$ множество $\{b, c\}$ покрывает все три ребра, а $\{b\}$ не покрывает cd. Два непересекающихся ребра ab и cd требуют двух различных выбранных вершин, поэтому минимум равен 2. Для звезды с центром v и сотней листьев достаточно $\{v\}$: большой граф может иметь маленькое покрытие.

Клика (clique) — множество попарно смежных вершин. Задача **Clique** спрашивает, есть ли клика размера k. Формулировка «не меньше k» эквивалентна: любое k-подмножество большей клики тоже клика. **Независимое множество (independent set)** — множество попарно несмежных вершин. S является покрытием тогда и только тогда, когда $V \setminus S$ независимо: в обоих случаях запрещено ребро с двумя концами вне S.

Ответ YES означает существование требуемого объекта, NO — его отсутствие. Алгоритм решения может возвращать только ответ; алгоритм поиска возвращает также сам объект. В примерах курса будем объяснять восстановление объекта.

1.3 Формальное определение параметризованной задачи

Параметризованная задача — множество пар $Q \subseteq \Sigma^* \times \mathbb{N}$, где Σ — фиксированный конечный алфавит, x — закодированные данные, k — неотрицательный целый параметр. Требуется проверить принадлежность $(x, k) \in Q$. Параметр не всегда бюджет: им могут быть число переменных формулы, число терминалов графа или ширина разложения.

Обозначим полную длину входа через N. Для формальных определений этого курса используем унарную запись параметра, то есть $N = |x| + k$ с точностью до служебных разделителей. В графовых задачах данные кодируют все вершины и рёбра; веса, если есть, входят в N своей двоичной записью. Нельзя считать число с миллионом бит одной бесплатной операцией.

В Vertex Cover при $k \geq n$ ответ YES, а в Clique при $k > n$ — NO. Поэтому содержательные входы этих задач можно ограничить $k \leq n$. При $k = 0$ пустое покрытие допустимо ровно для графа без рёбер; пустую клику считаем допустимой.

1.4 FPT: вся неполиномиальность зависит только от параметра

Задача принадлежит **FPT (fixed-parameter tractable)**, если существуют один детерминированный алгоритм A, вычислимая функция $f : \mathbb{N} \to \mathbb{N}$ и константа c, такие, что для каждого входа алгоритм даёт правильный ответ за время

$$T_A(x, k) \leq f(k)N^c.$$

Порядок кванторов существенен: сначала выбираются A, f, c, затем оценка действует для всех (x, k). Степень c не зависит от k. Функция f может расти быстро; определение выражает форму зависимости, а не обещание практической скорости.

«Один алгоритм» означает единообразие (uniformity): одна программа принимает любой k. Нельзя подменить её утверждением «для каждого k где-то существует подходящая программа», если не дан способ получать эти программы. Вычислимость f также входит в определение.

Например, $2^k N^3$, $k!N^2$ и $2^{2^k}N$ — FPT-оценки. Даже N^{100} подходит, хотя может быть непрактичным. Если увеличить N в десять раз, полиномиальный множитель растёт в 10^c раз независимо от параметра. У алгоритма N^k тот же рост равен 10^k.

1.5 XP: полиномиальность каждого фиксированного среза

Задача принадлежит **XP (slice-wise polynomial)**, если один алгоритм имеет оценку

$$T_A(x, k) \leq f(k)N^{g(k)}$$

для вычислимых f, g. Срезом при k_0 называем задачу только на входах с $k = k_0$. В каждом таком срезе степень $g(k_0)$ постоянна, но единой постоянной степени для всех параметров может не быть. Из определения сразу следует FPT $\subseteq$ XP: достаточно $g(k) = c$.

Оценку N^k нельзя ограничить $f(k)N^c$ для всех входных размеров. Предположим обратное. Зафиксируем целое $k_0 > c$. Тогда $N^{k_0-c} \leq f(k_0)$ для любого достаточно большого N. Правая часть постоянна, левая неограниченно растёт — противоречие. Аналогично для $N^{\log_2(k+1)}$ выбираем k_0 так, чтобы $\log_2(k_0 + 1) > c$.

Это рассуждение касается формы оценки. Если известен алгоритм времени N^k, то задача доказанно лежит в XP, но другой алгоритм может оказаться FPT. Более того, грубая верхняя оценка N^k сама по себе даже не доказывает, что первый алгоритм действительно работает столь долго.

1.6 Vertex Cover: от полного перебора к FPT

Прямой способ — перечислять подмножества вершин размера до k и для каждого просматривать рёбра. Для $1 \leq k \leq n$ кандидатов не больше $\sum_{i=0}^{k} n^i \leq (k + 1)n^k$, проверка занимает полиномиальное время. Получили XP.

Теперь выберем любое ребро uv. Всякое покрытие обязано выбрать хотя бы одну из вершин u, v. Поэтому достаточно проверить два случая: выбрать u или выбрать v. После выбора вершины все инцидентные ей рёбра уже покрыты; удаляем её из графа и уменьшаем бюджет на один. В каждом случае решаем ту же задачу на остатке.

Если бюджет стал отрицательным, этот случай невозможен. Если рёбер не осталось и бюджет неотрицателен, уже выбранные вершины образуют решение. В остальных случаях повторяем выбор ребра. Мы не знаем заранее правильный выбор, но перебираем оба; успешной должна быть хотя бы одна ветвь.

Почему это корректно. Для существующего покрытия хотя бы один конец uv лежит в нём; удаление этого конца даёт покрытие соответствующего остатка. Обратно, любое покрытие остатка вместе с выбранным концом покрывает исходный граф. Индукция по уменьшающемуся бюджету доказывает правильность рекурсии, включая отрицательную базу.

Почему это быстро по параметру. До исчерпания бюджета можно сделать не больше $k+1$ выборов. На глубине i не больше 2^i вызовов; всего не больше $\sum_{i=0}^{k+1} 2^i < 2^{k+2}$. Создание остатка и выбор ребра стоят $O(n+m)$, если каждый раз просто копировать граф. Получаем $O(2^k(n+m))$, то есть FPT. Записав выбранные вершины на успешном пути, получим покрытие. Следующая глава подробно объясняет этот метод и улучшает оценку.

NP-полнота не противоречит этому алгоритму: если k растёт вместе с n, множитель 2^k может быть экспоненциальным от размера графа. FPT не обещает полиномиальности при произвольном параметре.

1.7 Почему такой же аргумент не решает Clique

Для клики можно проверить все $\binom{n}{k}$ наборов за $O(n^k k^2)$ после подготовки проверки смежности. Но произвольное ребро не задаёт обязательного выбора конца: искомая клика может находиться в другой компоненте. Ветвление по произвольной вершине «взять / не взять» тоже не даёт оценки 2^k: отказ не уменьшает размер ещё искомой клики.

Это объясняет, почему конкретные попытки не дают FPT. Оно не является доказательством отсутствия FPT вообще. Для содержательной условной нижней оценки в курсе вводятся параметризованные редукции и W[1]-трудность.

1.8 Одна задача, разные параметры

CNF-SAT получает конъюнкцию дизъюнкций литералов и спрашивает, существует ли удовлетворяющее присваивание. Литерал — переменная или её отрицание. Если параметр p — число переменных, перебор 2^p присваиваний с проверкой формулы даёт $2^p N^{O(1)}$.

Если параметр q — максимальное число литералов в дизъюнкции, ситуация другая. При $q = 3$ получаем NP-полную 3-SAT. FPT-алгоритм по q работал бы на этом срезе за $f(3)N^c$, то есть полиномиально, и влёк бы $P = NP$. Здесь используется известная из базовой сложности NP-полнота 3-SAT.

Для нескольких параметров допустима совместная зависимость: $2^{kd}N^3$ — FPT по паре (k, d), которую можно заменить суммой $k + d$, поскольку $kd \le (k+d)^2$. Но эта оценка не гарантирует FPT по одному k, если d не ограничено функцией k.

1.9 Как читать обозначения времени

$O^*(f(k))$ означает $f(k)N^{O(1)}$; скрытая степень не зависит от параметра. Например, $O^*(2^k)$ может скрывать N^2, но не N^k. В задачах с весами полиномиальность по численному значению веса ещё не является полиномиальностью по входу: число W записывается за $O(\log(W+1))$ бит.

Различайте три утверждения: найден конкретный алгоритм; доказана принадлежность задачи классу; условно исключён определённый тип алгоритмов. Первые два следуют из верхней оценки, третье требует отдельного аргумента.

1.10 Самопроверка

1. Почему $f(10)N^{100}$ всё ещё полином для фиксированного параметра?

2. В каком месте доказательства для Vertex Cover использовано свойство покрытия, а не произвольного подмножества вершин?
3. Почему экспонента только от k допускает NP-полноту?
4. Что сломается при попытке заменить параметр числа переменных SAT длиной дизъюнкции?

1.11 Практика

Задачи 01.1–01.5. Задачи 01.4–01.5 добавлены для самостоятельного применения определений; разборы находятся отдельно.

1.12 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §1.1, с. 12–14. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.